

I. DESIGN

A. Chosen Components

1) *DCS Controller*: An Arduino MEGA 2560 Rev3 was chosen as the DCS for this project. Initially we intended on using an Arduino Uno Rev3 however the device did not contain enough memory for us to provide it with the generated code (by 4%).

2) *SCADA Supervisor*: A Raspberry Pi v4 is being used as the SCADA controller, however a significant portion of testing was done using a Framework 16 Laptop.

3) *Tubing*: Quarter inch thick, clear plastic tubing is used for all actuator connections where liquid is transported.

4) *Container/Project Platform*: A three compartment plastic divider is being used as the platform for this project. The middle compartment was removed and is being used as space to place the valves and flowrate sensors. Holes were placed in the bottom of the top container (the container that is being level controlled) with fittings added for hose connections.

5) *Pumps*: HiLetgo DC 12V water pumps were used to move water up (into the controlled vessel) and down (out of the controlled vessel).

6) *Valves*: 12V normally closed solenoid valves were used to stop siphoning when pumping water down, and to implement a manually controlled valve that allows water to be emptied from the upper container.

7) *Water-Level Sensor*: A ruler-style analog water level sensor is used to determine the water level of the container. This is the primary input for determining actuation.

8) *Temperature Sensor*: It was suggested that temperature would be a good metric to track inside our system, however due to scope reduction it was not implemented.

9) *Flowrate Sensor*: It was suggested that flowrate detection across our pumps would be a good metric to track inside our system allowing PID style control with feedback, however due to scope reduction it was not implemented.

B. Electronics

An Arduino is not typically capable of delivering sufficient current to actuation devices such as pumps and solenoids. To alleviate this issue, a 12V DC supply is used to power the Arduino (barrel connection) and a set of relays. These relays are what is used to drive the pumps and valves, the Arduino simply outputs (digital) to these devices at 5V. The Raspberry Pi is separately powered. The wiring was connected using Wago connectors, and utilized wires of various gauges.

C. Manual Switches

Several manual switches are present that allow a single valve to be manually controlled, and system power to be manually controlled. This is implemented using simple rocker-style switches.

D. Network

To isolate our network (for SSH and GUI connection purposes), a TP-Link nano-router was used as a network extender for my Laptop's hotspot. This allows any device connected to that hotspot to wirelessly command the SCADA system.

TABLE I
SENSORS CHOSEN FOR THE CONTROL SYSTEM.

Sensor	Item Name	Role	Implementation Details
Water Level Sensor	Robot Contact Water/Liquid Level Sensor	Detects if water in the tank reaches a high or low point. Prevents overfilling or running dry.	Optical sensor output (HIGH/LOW). Connect to GPIO input. Can trigger pump ON/OFF automatically.
Flow Rate Sensor	1/4" Water Flow Sensor (Hall-Effect)	Measures water flow rate in L/min to ensure the pump is moving water and to calculate total volume.	Generates digital pulses proportional to flow. Connect to GPIO input; count pulses using interrupt.
Temperature Sensor	NTC Thermistor 10 kΩ MF52-103	Measures the water temperature. Can detect overheating or environmental changes.	Analog sensor. Use an ADC module (e.g. MCP3008) to read voltage, then convert to temperature in code.

TABLE II
ACTUATORS USED IN THE CONTROL SYSTEM.

Actuator	Item Name	Role	Implementation Details
Water Pump	HiLetgo DC 12 V 240 L/h Micro Brushless Pump	Pumps water into or out of the tank. Controlled automatically based on level sensor input.	Connect through the Pump Controller (relay/MOSFET) that switches the 12 V power.
Hydraulic Valve	1/4" DC 12 V 2-Way Normally Closed Solenoid Valve (WIC Valve 2ACK Series)	Opens/closes to allow or stop water flow.	Controlled through the Valve Controller. When the coil is energized, the valve opens; otherwise, it closes.

TABLE III
MODULES INTERFACING THE RASPBERRY PI WITH ACTUATORS.

Controller	Connected To	Role	Implementation
Pump Controller	Water Pump	Acts as a switch driver—receives a 3.3 V signal from the Pi to toggle 12 V power to the pump.	Can be a relay board or a MOSFET circuit. Isolates the Pi from the high-power pump.
Valve Controller	Hydraulic Valve	Same principle as the Pump Controller, but for opening/closing the valve.	When Pi output is HIGH $\rightarrow$ coil energized $\rightarrow$ valve opens. When LOW $\rightarrow$ valve closes.

TABLE IV
SCADA / HUMAN–MACHINE INTERFACE COMPONENTS.

Component	Role	Description
Laptop / Computer	CLI or GUI	Acts as the human–machine interface (HMI) for remote monitoring and manual control.
Wireless Communication	Local Area Wifi	Connects the Raspberry Pi to the laptop using Wi-Fi. Can send real-time data or receive control commands.
Software Options	Data Visualization	Flask Web Server: Allows CLI or GUI to send commands to SCADA. Plotly: Allows GUI to poll SQL database to generate plots. SSH: Remote access for starting and troubleshooting server.

TABLE V
COMMUNICATION LINK OPTIONS BETWEEN THE RASPBERRY PI AND ARDUINO.

Link	Pros	Cons	Method
USB Serial (UART over USB) (Recommended)	Easiest; powered and isolated via cable; stable.	Requires a USB cable.	Connect Arduino to Pi via USB; use <code>/dev/ttyACM0</code> or <code>/dev/ttyUSB0</code> at 115200 baud.
I²C (Pi master, Arduino slave)	Simple 2-wire interface.	3.3 $\leftrightarrow$ 5 V level shifting; address management.	Pi SDA/SCL $\leftrightarrow$ Arduino A4/A5 (Uno); add bi-directional level shifter.
SPI	Fast throughput.	Chip-select management; level shifting required.	Pi SPI0 $\leftrightarrow$ Arduino SPI pins; level shifter required.
RS-485 (Modbus RTU)	Long cable runs; noise-robust.	Requires transceivers and protocol stack.	MAX485 modules, twisted-pair cable, Modbus library.

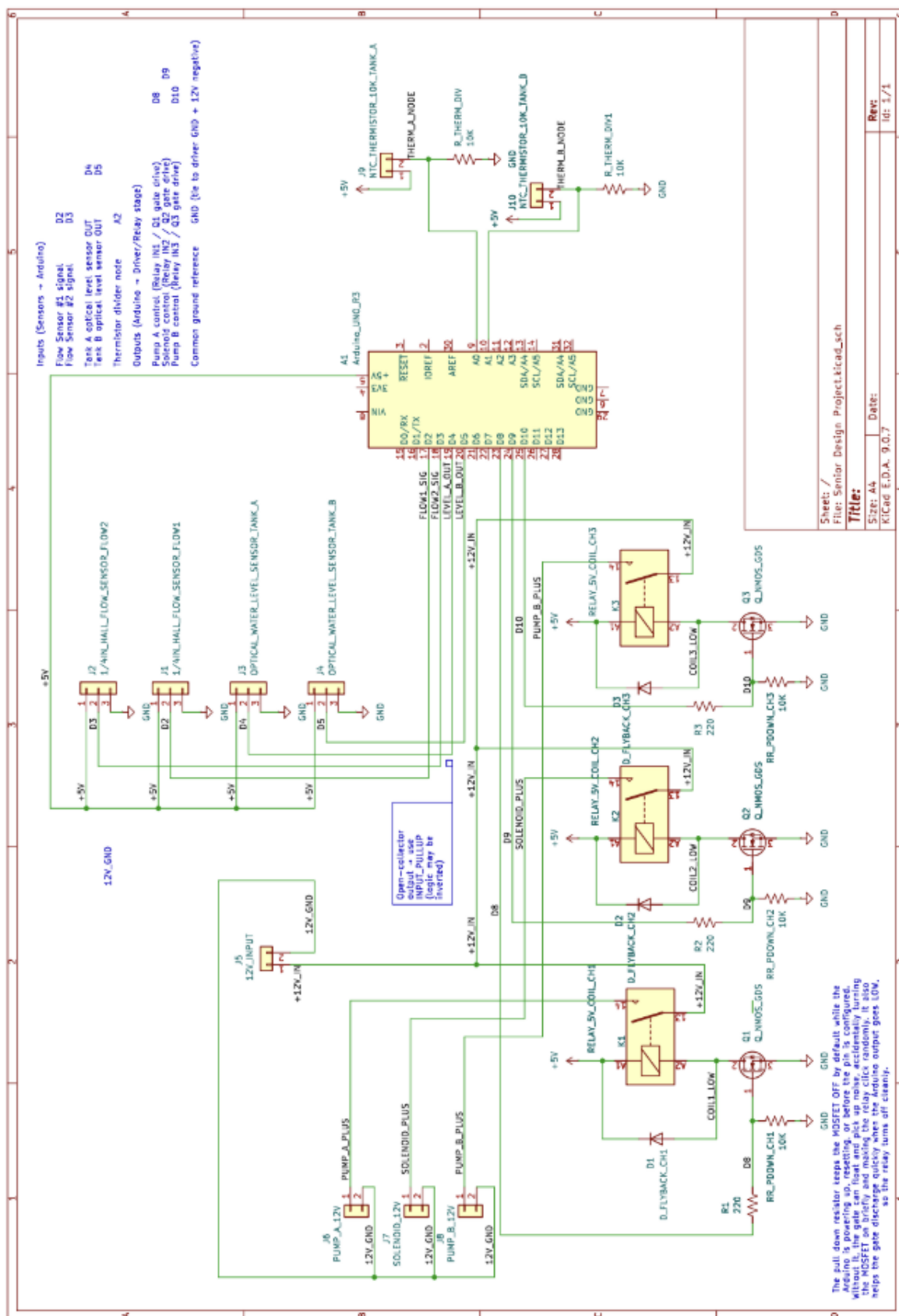


Fig. 1. Initial Hardware Schematic

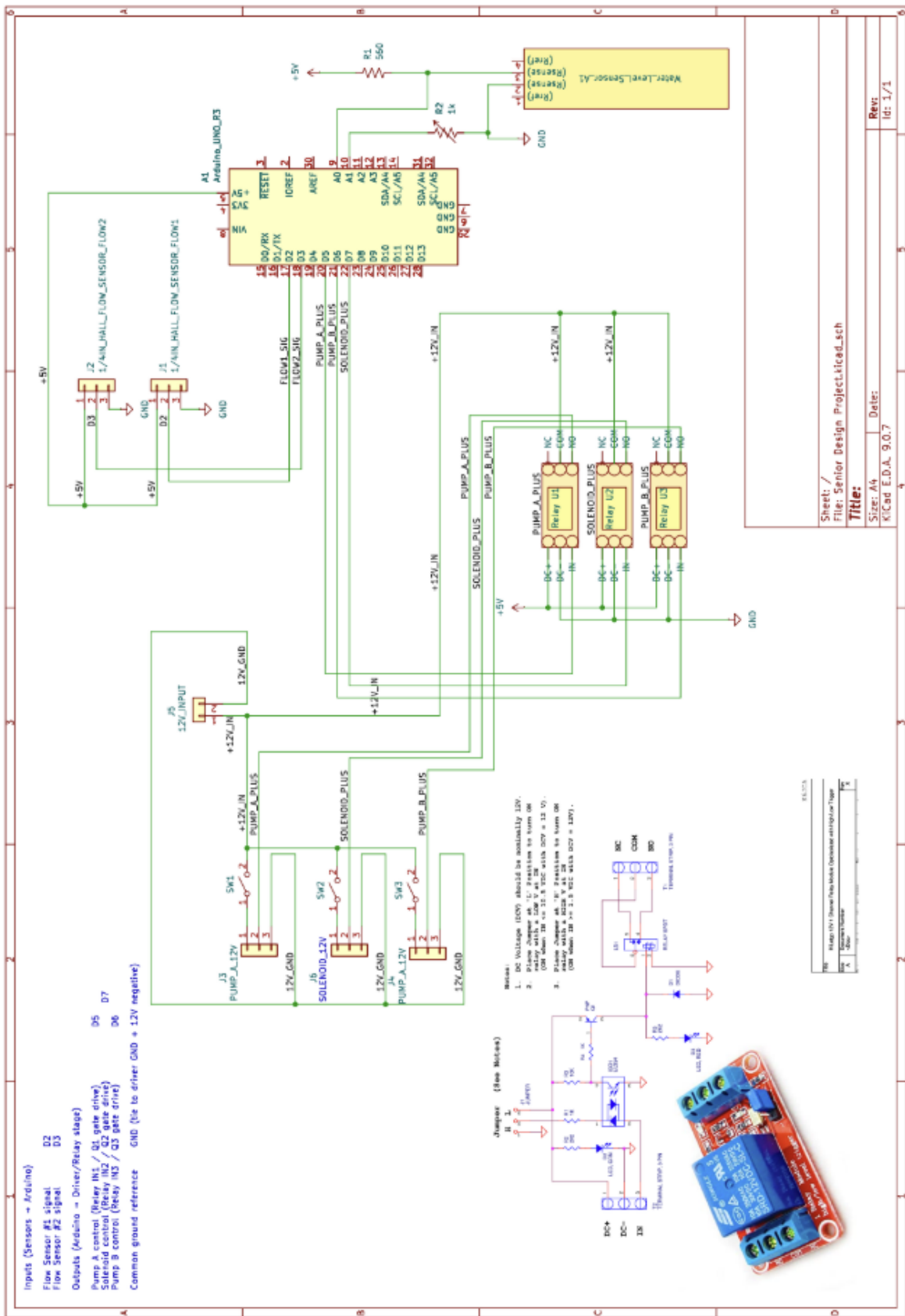


Fig. 2. Final Hardware Schematic

E. Software Design